

人工智能技术实验

Artificial Intelligence Technology Experiment

许丽

南开大学人工智能学院

2025年9月

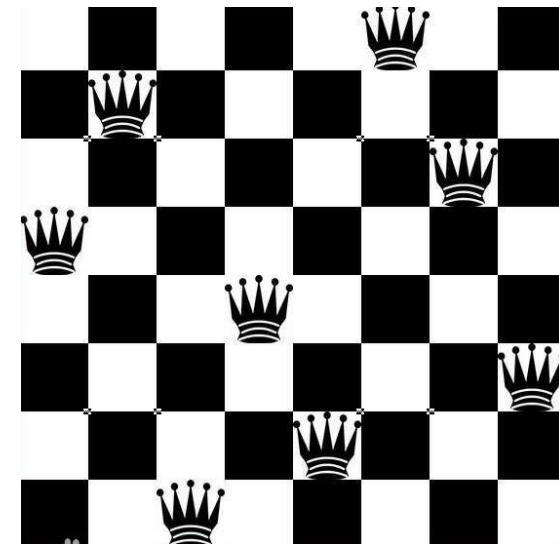




L2 八皇后问题

八皇后问题概述

- 八皇后问题是一个以国际象棋为背景的问题：如何能够在 8×8 的国际象棋棋盘上放置八个皇后，使得任何一个皇后都无法直接吃掉其他的皇后？为了达到此目的，任两个皇后都不能处于同一条横行、纵行或斜线上。
- 八皇后问题可以推广为更一般的N皇后摆放问题：这时棋盘的大小变为 $N \times N$ ，而皇后个数也变成N。



八皇后问题由来

- 八皇后问题，是一个古老而著名的问题，是回溯算法的典型用例。
- 最早是由国际西洋棋棋手马克斯·贝瑟尔于1848年提出。之后陆续有数学家对其进行研究，其中包括高斯和康托，并且将其推广为更一般的n皇后摆放问题。
- 高斯认为有76种方案。1854年在柏林的象棋杂志上不同的作者发表了40种不同的解，后来有人用图论的方法解出92种结果。计算机发明后，有多种计算机语言可以解决此问题。
- 八皇后问题出现在1990年代初期的著名电子游戏第七访客中。

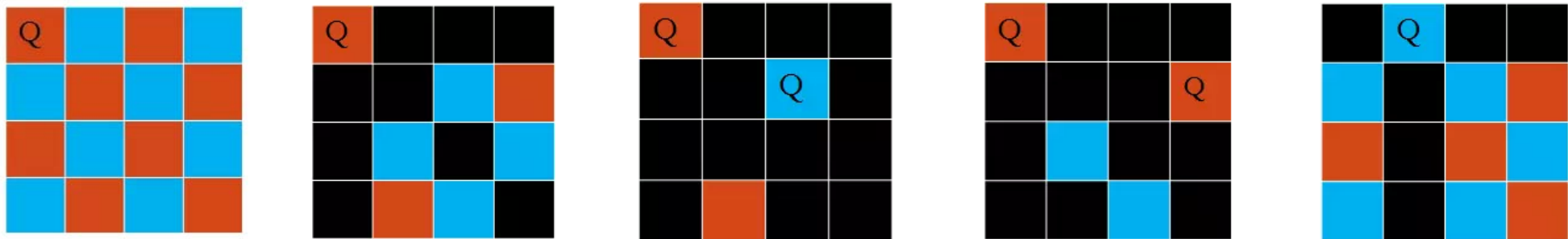


八皇后问题解法

- 穷举法：64取8 暴力解法 组合爆炸
- 加入规则：每行每列只能有一个皇后，8!
- 递归回溯法，试探法
 - 冲突：行、列、对角线

八皇后问题解析

- 递归方法求解：函数调用自身，只是参数大小有所改变。
- 能进则进，不能进则换，不能换则退。



确定第一个皇后位置→递归尝试下一个皇后位置→找到解决方案

八皇后问题约束条件

- 将第 i 个皇后的位置记为 (X_i, Y_i) , 其中 X_i 代表第 i 个皇后在棋盘上所处的行, Y_i 代表第 i 个皇后在棋盘上所处的列。
- 根据国际象棋中皇后的攻击线, 棋盘上的八个皇后中任意的两个皇后 (X_i, Y_i) 和 (X_j, Y_j) (其中, $1 \leq i, j \leq 8$) , 都必须满足所在行、列、正斜线、反斜线上没有皇后, 可表示为:
 - (1) $X_j \neq X_i$ (横行) (2) $Y_j \neq Y_i$ (纵列)
 - (3) $Y_j - X_j \neq Y_i - X_i$ (正斜线方向)
 - (4) $Y_j + X_j \neq Y_i + X_i$ (反斜线方向)

八皇后问题求解

- 数据初始化
 - 皇后位置：
 - 二维数组 `int gEightQueen[8][8]`
 - 一维数组 `int gEightQueen[8]`
- 攻击范围的表示方法：八个方向（占领状态）
- 八皇后问题中的核心代码：
 - 遍历过程函数：递归回溯法
 - 判断函数：判断哪些状态适合继续扩展，或者作为答案状态。

八皇后问题参考程序-C++

```
#include<iostream>
using namespace std;
static int gEightQueen[8] = { 0 }, gCount = 0;
void print()//输出每一种情况下棋盘上皇后的摆放情况
{
    for (int i = 0; i < 8; i++)
    {
        int inner;
        for (inner = 0; inner < gEightQueen[i]; inner++)
            cout << "0";
        cout << "#";
        for (inner = gEightQueen[i] + 1; inner < 8; inner++)
            cout << "0";
        cout << endl;
    }
    cout << "=====\n";
}
int check_pos_valid(int line, int list)//检查是否存在有多个皇后在同一行/列/对角线的情况
{
    int index;
    int data;
    for (index = 0; index < line; index++)
    {
        data = gEightQueen[index];
        if (list == data)
            return 0;
        if ((index + data) == (line + list))
            return 0;
        if ((index - data) == (line - list))
            return 0;
    }
    return 1;
}
```

```
void eight_queen(int line)
{
    int list;
    for (list = 0; list < 8; list++)
    {
        if (check_pos_valid(line, list))
        {
            gEightQueen[line] = list;
            if (7 == line)
            {
                gCount++, print();
                gEightQueen[line] = 0;
                return;
            }
            eight_queen(line + 1);
            gEightQueen[line] = 0;
        }
    }
}
int main()
{
    eight_queen(0);
    cout << "total=" << gCount << endl;
    return 0;
}
```

八皇后问题参考程序-python

判断函数

```
def is_safe(board, row, col):
    # 检查当前位置的上方是否有皇后
    for i in range(row):
        if board[i] == col:
            return False
    # 检查当前位置的左上方是否有皇后
    if board[i] == col - (row - i):
        return False
    # 检查当前位置的右上方是否有皇后
    if board[i] == col + (row - i):
        return False
    return True
```

递归函数

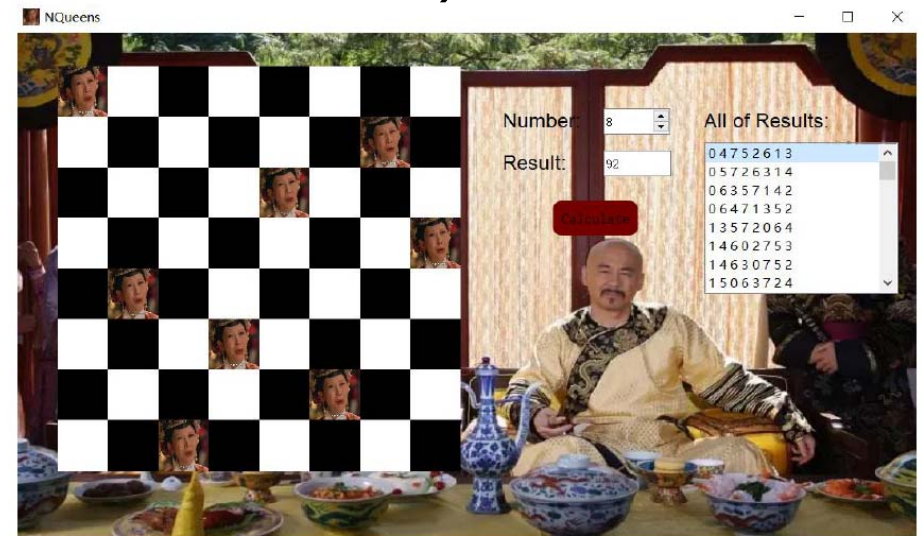
```
def solve_n_queens(board, row):
    if row == len(board):
        # 打印出解法
        for i in range(len(board)):
            for j in range(len(board)):
                if board[i] == j:
                    print("Q ", end="")
                else:
                    print(".", end="")
            print()
        print()
    else:
        for col in range(len(board)):
            if is_safe(board, row, col):
                board[row] = col
                solve_n_queens(board, row + 1)
```

棋盘输出

```
board = [0] * 8
solve_n_queens(board, 0)
```

八皇后问题实验要求

- 基本要求：实现**N**皇后问题的解法统计（90分）
- 选做内容：界面（10分）
- 提交时间：二周
- 提交方式：在线提交（**实验报告、源代码、demo等**）
- 迟交问题：扣分
- 抄袭问题：不及格处理！



多项选择题

题目：八皇后之间需满足的条件

- A.不在同一行上
- B.不在同一列上
- C.不在同一正斜线上
- D.不在同一反斜线上

S (C
VJ C P M U!

